

Создание ролей пользователей с разными возможностями в MS SQL Server

1. Подготовка базы данных

Откройте **SQL Server Management Studio** и выполните скрипт от имени администратора:

```
CREATE DATABASE PracticeSecurityDB;  
GO
```

```
USE PracticeSecurityDB;  
GO
```

```
CREATE TABLE Employees (  
    EmployeeID INT IDENTITY PRIMARY KEY,  
    FullName NVARCHAR(100),  
    Position NVARCHAR(100),  
    Salary MONEY  
);
```

```
INSERT INTO Employees (FullName, Position)  
VALUES  
(N'Иванов Иван', N'Менеджер'),  
(N'Петров Петр', N'Бухгалтер'),  
(N'Сидорова Анна', N'Директор');  
GO
```

2. Создание логинов и пользователей

2.1. LOGIN и USER

В MS SQL Server есть **двухуровневая модель безопасности**

1. LOGIN — уровень сервера

- Это учетная запись для **подключения к серверу**
- Проверяется при входе (аутентификация)
- Хранится в базе master

2. USER — уровень базы данных

- Это учетная запись внутри конкретной БД
- Определяет **права доступа к таблицам**
- Связывается с LOGIN

после подключения каждому пользователю ставится в соответствие учетная запись (Login), а затем — пользователь базы данных

2.2. Создание LOGIN (вход на сервер)

Выполняется на уровне сервера:

```
CREATE LOGIN reader_login WITH PASSWORD = 'Reader_12345';  
CREATE LOGIN editor_login WITH PASSWORD = 'Editor_12345';  
CREATE LOGIN admin_login WITH PASSWORD = 'Admin_12345';  
GO
```

Команда	Описание
CREATE LOGIN	создаёт учетную запись на сервере
PASSWORD	задаёт пароль

reader_login	имя логина
--------------	------------

LOGIN сам по себе не даёт доступ к базе данных. **Пользователь сможет подключиться к серверу, но не работать с БД**

2.4. Создание USER (доступ к БД)

```
CREATE USER reader_user FOR LOGIN reader_login;
CREATE USER editor_user FOR LOGIN editor_login;
CREATE USER admin_user FOR LOGIN admin_login;
GO
```

Часть	Значение
CREATE USER	создаёт пользователя в БД
reader_user	имя пользователя в БД
FOR LOGIN	связывает с логином
reader_login	существующий LOGIN

2.5. Связь LOGIN → USER

После выполнения:

reader_login (сервер)

↓

reader_user (БД)

Только после этого пользователь:

- может **войти на сервер**
- может **работать в конкретной БД**

2.6. Проверка пользователей БД

```
SELECT name, type_desc
FROM sys.database_principals
WHERE type IN ('S', 'U');
```

или:

```
EXEC sp_helpuser;
```

3. Создание ролей базы данных

3.1. Что такое роль?

Роль — это именованный объект БД, которому можно назначать привилегии и затем назначать её пользователям

Проще:

- роль = **группа прав**
- пользователь = **член роли**

3.2. Зачем нужны роли?

Без ролей:

User1 → права

User2 → права

User3 → права

С ролями:

Role → права

Users → входят в роль

Роль упрощает администрирование и снижает ошибки пользователей с одинаковыми правами следует объединять в группы

3.3. Виды ролей в SQL Server

1. Фиксированные роли БД (уже есть в системе)

Примеры:

- db_datareader — только чтение
- db_datawriter — запись
- db_owner — полный доступ

Их менять нельзя

2. Пользовательские роли (наш случай)

Создаются вручную:

```
CREATE ROLE role_reader;
```

Мы создаём **свои роли под бизнес-логику**

3.4. Создание ролей

```
CREATE ROLE role_reader;
```

```
CREATE ROLE role_editor;
```

```
CREATE ROLE role_admin;
```

```
GO
```

Команда	Значение
CREATE ROLE	создаёт роль
role_reader	имя роли
GO	разделитель пакетов

3.5. Логика ролей

Роль	Назначение
role_reader	только чтение
role_editor	чтение + изменение
role_admin	полный доступ

3.6. Создание роли с владельцем

Можно указать владельца:

```
CREATE ROLE role_manager AUTHORIZATION dbo;
```

- dbo — владелец роли
- владелец может управлять ролью

3.7. Проверка ролей

```
SELECT name, type_desc  
FROM sys.database_principals  
WHERE type = 'R';
```

или:

```
EXEC sp_helprole;
```

Покажет список ролей. После создания CREATE ROLE role_reader роль не имеет прав и не имеет пользователей. Это просто "контейнер"

Роль участвует в цепочке LOGIN → USER → ROLE → ПРАВА → ОБЪЕКТЫ

После создания можно:

- Добавлять пользователей
- Назначать права

- Запрещать
- Удалять

4. Добавление пользователей в роли (очень подробно)

4.1. Что происходит на этом этапе?

После предыдущих шагов у нас есть:

- LOGIN (вход на сервер)
- USER (доступ к БД)
- ROLE (контейнер прав)

Но пока **они не связаны между собой**

4.2. Логическая схема

До добавления в роль USER → (нет прав), ROLE → (нет пользователей)

После: USER → ROLE → ПРАВА → ТАБЛИЦЫ. Это ключевая идея всей системы безопасности

4.3. Основная команда

```
ALTER ROLE role_reader ADD MEMBER reader_user;
ALTER ROLE role_editor ADD MEMBER editor_user;
ALTER ROLE role_admin ADD MEMBER admin_user;
GO
```

Часть	Значение
ALTER ROLE	изменить роль
role_reader	имя роли
ADD MEMBER	добавить участника
reader_user	пользователь

Что происходит внутри SQL Server? Когда выполняется ALTER ROLE role_reader ADD MEMBER reader_user;

SQL Server:

1. Находит роль role_reader
2. Находит пользователя reader_user
3. Добавляет запись в системную таблицу sys.database_role_members

4.4. Проверка членства

SELECT

```
    r.name AS RoleName,
    u.name AS UserName
```

FROM sys.database_role_members rm

JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id

JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id;

Покажет:

role_reader → reader_user

role_editor → editor_user

role_admin → admin_user

4.5. Один пользователь может быть в нескольких ролях

```
ALTER ROLE role_reader ADD MEMBER reader_user;
```

```
ALTER ROLE role_editor ADD MEMBER reader_user;
```

Теперь reader_user → role_reader + role_editor

4.6. Добавление нескольких пользователей

```
ALTER ROLE role_reader ADD MEMBER user1;  
ALTER ROLE role_reader ADD MEMBER user2;  
ALTER ROLE role_reader ADD MEMBER user3;
```

Роль = группа пользователей

4.7. Удаление пользователя из роли

```
ALTER ROLE role_reader DROP MEMBER reader_user;
```

Пользователь:

- остаётся в БД
- но теряет права роли

4.8. Проверка через системные процедуры

```
EXEC sp_helprolemember 'role_reader';
```

Покажет всех участников роли

5. Назначение прав ролям (GRANT / DENY / REVOKE)

5.1. Теория

Привилегии — это операции, которые разрешено выполнять пользователю над объектами БД

Основные операции:

- SELECT — чтение
- INSERT — добавление
- UPDATE — изменение
- DELETE — удаление

5.2. Общая схема

ROLE → ПРАВА → ОБЪЕКТ (таблица)

USER → ROLE

Пользователь получает права **через роль**

5.3. Команда GRANT (разрешение)

Синтаксис: GRANT <права> ON <объект> TO <роль>;

Пример (только чтение)

```
GRANT SELECT ON Employees TO role_reader;  
GO
```

- разрешаем читать таблицу Employees
- всем, кто в роли role_reader

Пользователь может: SELECT * FROM Employees;

Но не может: INSERT, UPDATE, DELETE

5.4. Роль редактора (несколько прав)

```
GRANT SELECT, INSERT, UPDATE ON Employees TO role_editor;  
GO
```

Право	Что делает
SELECT	читать
INSERT	добавлять строки
UPDATE	изменять строки

Проверка:

```
SELECT * FROM Employees;
```

INSERT INTO Employees VALUES (...);

UPDATE Employees SET Salary = 80000 WHERE EmployeeID = 1;

5.5. Команда DENY (запрет)

DENY DELETE ON Employees TO role_editor;
GO

запрет (DENY) имеет приоритет над разрешением (GRANT). Даже если есть:

GRANT DELETE ON Employees TO role_editor;

DENY DELETE ON Employees TO role_editor;

итог:

DELETE запрещён

DELETE FROM Employees WHERE EmployeeID = 1;

Ошибка: DELETE permission denied

5.6. Роль администратора (полные права)

GRANT SELECT, INSERT, UPDATE, DELETE ON Employees TO role_admin;
GO

Что получает admin: читать, добавлять, изменять, удалять

Это аналог db_owner, но только для таблицы

5.7. Внутренний механизм (как это хранится)

SQL Server сохраняет права в системных таблицах:

SELECT *

FROM sys.database_permissions;

Там видно:

- кому
- на что
- какие права

5.8. Комбинация прав

Пример:

GRANT SELECT ON Employees TO role_reader;

GRANT UPDATE ON Employees TO role_editor;

Если пользователь в обоих ролях:

итог: SELECT и UPDATE

5.9. Конфликт прав

Пример:

GRANT SELECT ON Employees TO role_reader;

DENY SELECT ON Employees TO role_editor;

Если пользователь в обоих ролях: SELECT запрещён

Почему так? Лучше запретить лишнее, чем случайно дать доступ

5.10. Ограничение по столбцам

Можно дать доступ не ко всей таблице:

GRANT SELECT (FullName, Position) ON Employees TO role_reader;

Пользователь: видит имя и должность и не видит зарплату

5.11. Ограничение через представление

Создаём представление:

```
CREATE VIEW Employees_Public AS  
SELECT FullName, Position FROM Employees;
```

Даём доступ:

```
GRANT SELECT ON Employees_Public TO role_reader;
```

Это более безопасно

5.12. Команда REVOKE (отмена)

```
REVOKE INSERT ON Employees FROM role_editor;
```

Команда	Что делает
GRANT	даёт право
DENY	запрещает
REVOKE	убирает (возвращает "неопределено")

- если нет GRANT → доступ запрещён
- REVOKE ≠ DENY

5.13. Практический сценарий

Было:

```
GRANT INSERT ON Employees TO role_editor;
```

Убрали:

```
REVOKE INSERT ON Employees FROM role_editor;
```

Теперь: вставка запрещена (потому что нет разрешения)

5.14. Проверка прав

```
EXEC sp_helprotect;
```

Покажет кто и какие права имеет

Итоговые запросы на создание Ролей и пользователей

1. Подготовка базы данных

Откройте **SQL Server Management Studio** и выполните скрипт от имени администратора:

```
CREATE DATABASE PracticeSecurityDB;
GO

USE PracticeSecurityDB;
GO

CREATE TABLE Employees (
    EmployeeID INT IDENTITY PRIMARY KEY,
    FullName NVARCHAR(100),
    Position NVARCHAR(100),
    Salary MONEY
);

INSERT INTO Employees (FullName, Position, Salary)
VALUES
(N'Иванов Иван', N'Менеджер', 60000),
(N'Петров Петр', N'Бухгалтер', 70000),
(N'Сидорова Анна', N'Директор', 120000);
GO
```

2. Создание логинов и пользователей

```
CREATE LOGIN reader_login WITH PASSWORD = 'Reader_12345';
CREATE LOGIN editor_login WITH PASSWORD = 'Editor_12345';
CREATE LOGIN admin_login WITH PASSWORD = 'Admin_12345';
GO

USE PracticeSecurityDB;
GO

CREATE USER reader_user FOR LOGIN reader_login;
CREATE USER editor_user FOR LOGIN editor_login;
CREATE USER admin_user FOR LOGIN admin_login;
GO
```

3. Создание ролей базы данных

```
CREATE ROLE role_reader;
CREATE ROLE role_editor;
CREATE ROLE role_admin;
GO
```

4. Добавление пользователей в роли

```
ALTER ROLE role_reader ADD MEMBER reader_user;
ALTER ROLE role_editor ADD MEMBER editor_user;
ALTER ROLE role_admin ADD MEMBER admin_user;
GO
```

5. Назначение прав ролям

Роль только для чтения

```
GRANT SELECT ON Employees TO role_reader;  
GO
```

Роль для чтения и изменения данных

```
GRANT SELECT, INSERT, UPDATE ON Employees TO role_editor;  
DENY DELETE ON Employees TO role_editor;  
GO
```

Роль администратора таблицы

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Employees TO role_admin;  
GO
```

6. Проверка прав пользователей

Выполните вход под пользователем reader_login и попробуйте:

```
USE PracticeSecurityDB;  
  
SELECT * FROM Employees;  
  
INSERT INTO Employees (FullName, Position, Salary)  
VALUES (N'Тестовый пользователь', N'Стажер', 30000);
```

Ожидаемый результат: SELECT выполнится, INSERT будет запрещен.

Выполните вход под пользователем editor_login:

```
USE PracticeSecurityDB;  
  
SELECT * FROM Employees;  
  
UPDATE Employees  
SET Salary = 75000  
WHERE FullName = N'Петров Петр';  
  
DELETE FROM Employees  
WHERE FullName = N'Иванов Иван';
```

Ожидаемый результат: SELECT и UPDATE выполнятся, DELETE будет запрещен.

Выполните вход под пользователем admin_login:

```
USE PracticeSecurityDB;  
  
DELETE FROM Employees  
WHERE FullName = N'Иванов Иван';
```

Ожидаемый результат: удаление выполнится.

7. Ограничение доступа к отдельному столбцу

Запретим роли role_reader просмотр зарплаты:

```
DENY SELECT ON Employees(Salary) TO role_reader;  
GO
```

Проверка:

```
SELECT FullName, Position FROM Employees;  
SELECT Salary FROM Employees;
```

Ожидаемый результат: первый запрос выполнится, второй будет запрещен.

8. Отмена прав

Отменим право на добавление данных для роли редактора:

```
REVOKE INSERT ON Employees FROM role_editor;  
GO
```

Проверка:

```
INSERT INTO Employees (FullName, Position, Salary)  
VALUES (N'Новый сотрудник', N'Оператор', 40000);
```

Ожидаемый результат: вставка больше недоступна.

9. Просмотр ролей и пользователей

```
EXEC sp_helprole;  
EXEC sp_helprolemember;  
EXEC sp_helpuser;  
GO
```

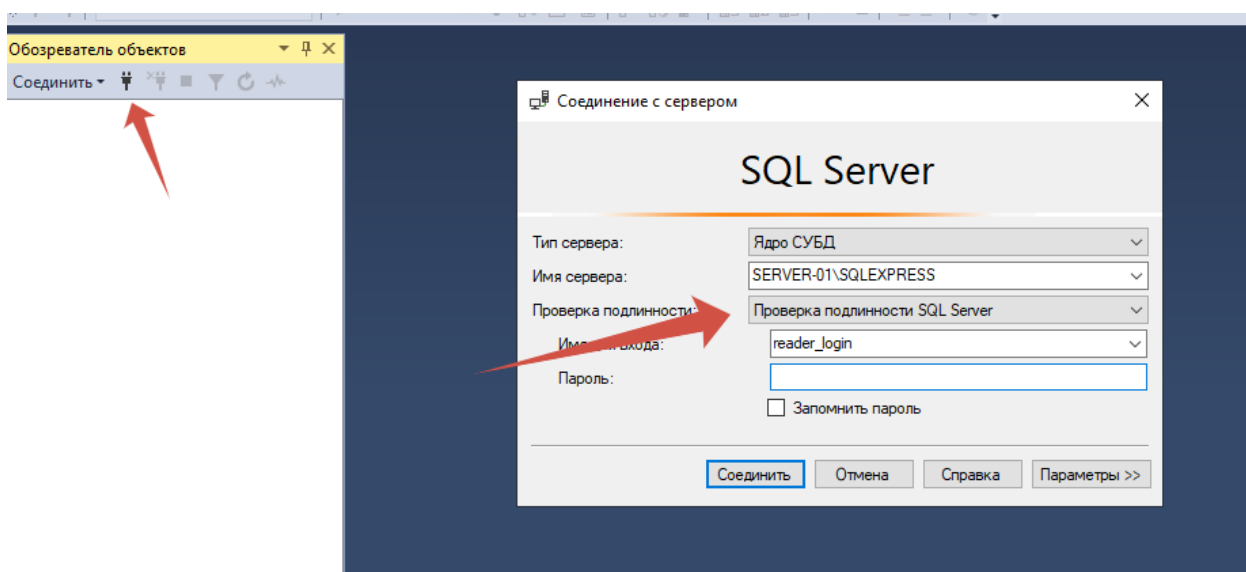
Как выполнить вход под другим пользователем (reader_login, editor_login, admin_login)

1. Открой **SQL Server Management Studio**
2. В верхнем меню нажми Соединить
3. В появившемся окне выбери:

Authentication **SQL Server Authentication**

Login reader_login

Password Reader_12345



Если не пускает значит не включена смешанная аутентификация

1. ПКМ по серверу → **Properties**
2. Вкладка **Безопасность**
3. Выбрать: Проверка подлинности SQL Server и Windows
4. Перезапустить SQL Server

3 АДМИНИСТРИРОВАНИЕ БАЗ ДАННЫХ И СЕРВЕРОВ

3.1 Администрирование серверов баз данных

В ходе выполнения работы была реализована система разграничения доступа в СУБД MS SQL Server. Были созданы база данных и таблица, содержащая тестовые данные, после чего настроены учетные записи пользователей на уровне сервера (LOGIN) и соответствующие им пользователи базы данных (USER).

Для управления доступом были созданы пользовательские роли, каждая из которых получила определённый набор привилегий. Назначение прав выполнялось с использованием операторов GRANT, DENY и REVOKE, что позволило реализовать различные уровни доступа:

- роль **role_reader** — только чтение данных;
- роль **role_editor** — чтение и изменение данных, но с запретом удаления;
- роль **role_admin** — полный доступ к данным.

(Запросы и скриншоты выполнения)

Назначение прав через роли значительно упрощает администрирование системы и позволяет централизованно управлять доступом пользователей к объектам базы данных.

Дополнительно ролям базы данных были назначены права доступа к таблице с использованием операторов GRANT, DENY и REVOKE. Оператор GRANT предоставляет разрешения на выполнение операций, DENY устанавливает запрет, имеющий приоритет над разрешениями, а REVOKE отменяет ранее назначенные права. Назначение прав через роли позволяет централизованно управлять доступом пользователей к объектам базы данных.

(Запросы и скриншоты выполнения)